

```
import React, {
  useRef,
  useEffect,
  forwardRef,
  useImperativeHandle,
  useCallback,
} from 'react';
import {
  Bold, Italic, Underline, Heading1, Heading2, Type,
  AlignLeft, AlignCenter, AlignRight, AlignJustify,
} from 'lucide-react';
import {
  useEditor, EditorContent, Node, mergeAttributes, ReactNodeViewRenderer, NodeViewWrapper,
} from '@tiptap/react';
import StarterKit from '@tiptap/starter-kit';
import UnderlineExtension from '@tiptap/extension-underline';
import TextAlign from '@tiptap/extension-text-align';
```

```
const PillView = ({ node, deleteNode }: any) => (  
  
  contentEditable={false}  
  style=  
    >  
    {node.attrs.name}  
  
  onClick={deleteNode}  
  style=  
  onMouseEnter={e => (e.currentTarget.style.background = '#bfdbfe')}  
  onMouseLeave={e => (e.currentTarget.style.background = 'transparent')}  
  
  >×  
  
);  
  
const VariablePill = Node.create({  
  name: 'variablePill',  
  group: 'inline',  
  inline: true,  
  atom: true,  
  selectable: true,  
  draggable: false,  
  addAttributes() {  
    return {  
      code: { default: " " },  
      name: { default: " " },  
    };  
  },  
  parseHTML() {  
    return [{  
      tag: 'span[data-variable]',  
      getAttrs: dom => ({  
  
        code: (dom as HTMLElement).getAttribute('data-variable'),  
        name: (dom as HTMLElement).getAttribute('data-name'),  
  
      }},  
    ]};  
  },  
  renderHTML({ node }) {  
    return [  
      'span',  
      mergeAttributes({  
        'data-variable': node.attrs.code || "",  
        'data-name': node.attrs.name || "",  
        class: 'variable-pill',  

```

```
contenteditable: 'false',
}),
node.attrs.name || ",
];
},
addNodeView() {
return ReactNodeViewRenderer(PillView);
},
});

// ADDED: PageBreak custom node
// Renders as a visible dashed line in the editor.
// Serialises to
```

in getHTML()

// so pdfUtils can detect it and call doc.addPage().

const PageBreakView = ({ deleteNode }: any) => (

contentEditable={false}

style=

>

background: 'white',

padding: '0 8px',

color: '#94a3b8',

fontSize: '11px',

fontFamily: 'sans-serif',

position: 'relative',

top: '-10px',

userSelect: 'none',

}}>

-- Page Break --

onClick={deleteNode}

style=

title="Remove page break"

>×

);

const PageBreak = Node.create({

name: 'pageBreak',

group: 'block',

atom: true,

selectable: true,

```
draggable: false,  
parseHTML() {  
  return [{ tag: 'div.page-break' }];  
},  
renderHTML() {  
  return ['div', {  
    class: 'page-break',  
    'data-page-break': 'true',  
    style: 'page-break-after:always;',  
  }];  
},
```

```
addNodeView() {  
  return ReactNodeViewRenderer(PageBreakView);  
},  
});
```

```
export interface SmartEditorRef {  
  getMarkdown: () => string;  
  setMarkdown: (markdown: string) => void;  
  
  insertVariable: (code: string, name: string) => void;  
  
  applyFormat: (command: string, value?: string) => void;  
}
```

```
interface SmartEditorProps {  
  initialMarkdown?: string;  
  onChange?: (markdown: string) => void;  
  placeholders?: Record;  
  className?: string;  
  placeholderText?: string;  
  showToolbar?: boolean;  
}
```